

STAT3009 / MACHINE LEARNING

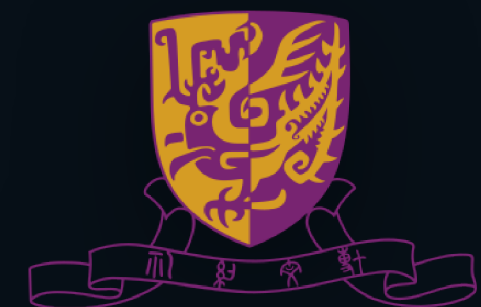
Machine Learning I: Models and Estimators

From prediction rules to reusable code

Ben Dai

CUHK · Department of Statistics and Data Science





WE CAN ALREADY PREDICT

global mean • user mean • item mean

How do we move from hand-designed averages to a general way of learning prediction rules from data?

BEFORE

Choose a formula

Then calculate its averages from the training triples.

→

TODAY

Choose a model family

Then learn its parameters from examples.

→

THE REAL CHALLENGE

Generalize

Work well on ratings the model has never seen.



The learning loop

3

01 • TRAIN

Examples with answers

features X_{train} + targets y_{train}

fit

02 • LEARN

Fitted model

$\hat{f}$ stores patterns learned from training data

predict

03 • APPLY

New examples

$X_{new} \rightarrow \hat{y}$

compare

04 • EVALUATE

Generalization

compare $\hat{y}$ with untouched answers



X_{test} is available. y_{test} stays out of fitting and model selection.



ML I • A language for prediction

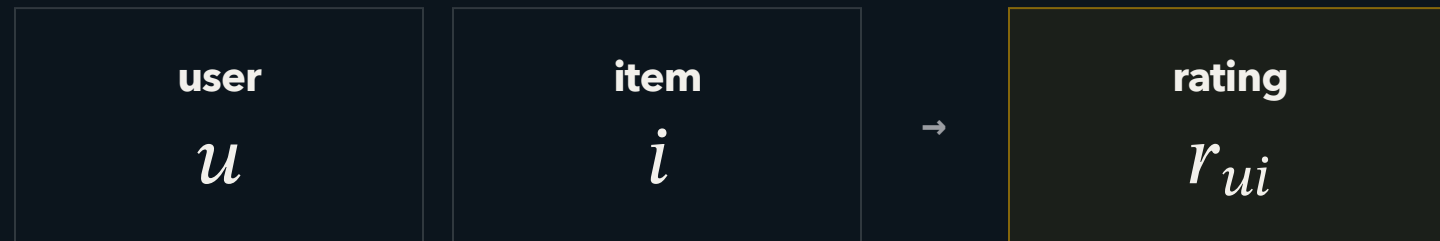
Examples, models, losses, parameters—and the separation between fitting and evaluation.



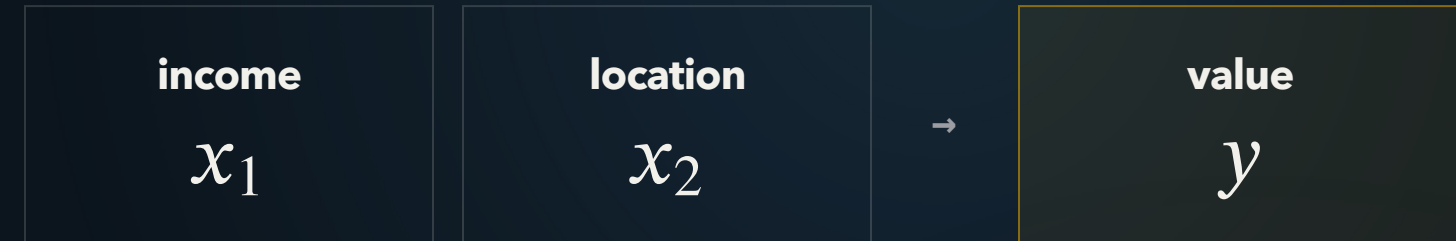
Rating prediction is supervised learning

5

RECOMMENDER SYSTEM



HOUSING REGRESSION



Supervised learning: learn a mapping from inputs to a known numerical target, then apply it to new inputs.



One row becomes one learning example

6

RATING DATA

user	item	rating
1960	670	4
1346	152	4
785	1741	4

X contains the inputs; y contains the answer.

ONE EXAMPLE

$$x_1 = (1960, 670)$$

the user-item pair

paired with

$$y_1 = 4$$

the observed rating

The row number is not a feature. It only identifies an observation.



Four questions decode every estimator

7

1 · MODEL

$$f_{\theta,h}(x)$$

What family of prediction rules can this estimator represent?

2 · PARAMETERS

$$\theta$$

What quantities will `fit` learn from the training data?

3 · HYPERPARAMETERS

$$h$$

What choices are fixed before `fit` and control the model or fitting rule?

4 · LOSS / CRITERION

$$L(y, \hat{y})$$

What does the fitting procedure regard as a better solution?

[Math Processing Error]



Parameters are learned; hyperparameters are chosen

8

BEFORE

fit

• CHOSEN

hyperparameter h

fit_intercept=True

Set by us—or later selected with validation—to control the model or fitting process.

`fit(X, y)`

uses training data

DURING

fit

• LEARNED

parameter $\hat{\theta}$

coef_ · intercept_

Estimated from the observed examples and stored in the fitted object.

One word, two conventions

scikit-learn calls constructor arguments “parameters” in `get_params()`. In our ML template, we call model settings **hyperparameters** to distinguish them from quantities learned during `fit`.

Quick test: **did the data determine the value?** Yes → learned parameter. No, fixed before fitting → hyperparameter.



A model is a family—not one fitted line

9

LINEAR FAMILY

$$f_{\theta}(x) = \theta_0 + \theta_1 x$$

The formula gives a family. Each value of $\theta = (\theta_0, \theta_1)$ gives a different member.



Training decides which member of the family to use.



Training minimizes average loss

10

1 · SCORE ONE PARAMETER SETTING

Fix $\theta^{(B)}$. For every training row, predict $\hat{y}_i = f_{\theta^{(B)}}(x_i)$ and calculate $\ell_i = (y_i - \hat{y}_i)^2$.

row i	observed y_i	prediction $\hat{y}_i$	loss ℓ_i
1	4.0	3.5	0.25
2	2.0	2.5	0.25
3	5.0	4.0	1.00

AVERAGE THE ROW LOSSES

[Math Processing Error]

2 · COMPARE PARAMETER SETTINGS

$$\theta^{(A)} \quad J(\theta^{(A)}) = 2.02$$

$$\theta^{(B)} \quad J(\theta^{(B)}) = 0.50$$

$$\hat{\theta} \quad J(\hat{\theta}) = 0.23$$

$$\hat{\theta} = \arg \min_{\theta} J(\theta)$$

minimum

the smallest loss value

arg minimum

the parameter value that achieves it

For squared loss, minimizing MSE also minimizes RMSE. A small training loss still does not guarantee good predictions on new rows.



Prediction freezes the fitted rule

11

AFTER FITTING

$$\hat{f}(x) = f_{\hat{\theta}}(x)$$

$\hat{\theta}$ is now fixed.

apply

NEW INPUT

x_{new}

features only

→

PREDICTION

$$\hat{y}_{new} = \hat{f}(x_{new})$$

`.fit(X_train, y_train)`

changes the model by learning parameters

`.predict(X_new)`

uses the fitted model without learning from new labels



TRAINING SET

How should the model fit?

- X_{train} and y_{train} are available
- estimates parameters
- may be reused during fitting

TEST INPUT AND LABEL

What may the model see?

- X_{test} is available for prediction
- y_{test} does not estimate parameters
- y_{test} is opened only for final evaluation

Generalization is performance on new examples drawn from the task we care about.



AVAILABLE FROM THE START

Inputs for fitting and prediction

X_{train}

training inputs

y_{train}

training answers

X_{test}

test user-item pairs

HIDDEN ANSWERS

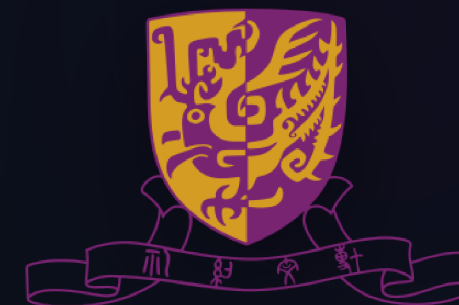
y_{test}

opened only after all modeling choices are fixed

MODEL SELECTION QUESTION

How can we compare model A and model B while y_{test} remains hidden?

ML II introduces holdout validation and cross-validation.



A complete regression example

California housing: define X and y , fit linear regression, predict, and calculate RMSE.



California housing is a regression task

ROWS

20,640

California block groups

FEATURES

8

demographic and geographic measurements

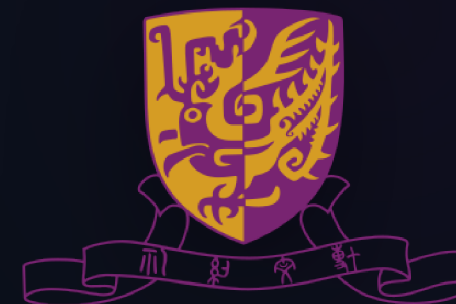
TARGET

MedHouseVal

median house value in units of \$100,000

MedInc HouseAge AveRooms AveBedrms Population AveOccup Latitude Longitude

Source: [scikit-learn California housing dataset documentation](#).



X is a matrix; y is a vector

FEATURE MATRIX

$$X_{train} \in \mathbb{R}^{13828 \times 8}$$

one row = one block group
one column = one feature

paired row by
row

TARGET VECTOR

$$y_{train} \in \mathbb{R}^{13828}$$

4.526
3.585
3.521
3.413

one numerical answer
for every row of X

`X_train.shape` (13828, 8) `y_train.shape` (13828,)



$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_8 x_8$$

 θ_0 **intercept**

the model's reference level

 θ_j **coefficient**

how prediction changes with feature j , holding others fixed

 $\hat{y}$ **prediction**

a weighted sum calculated for one row

The model family is linear. The coefficients are learned from the training data.



Fitting makes the training residuals small



ONE RESIDUAL

$$e_i = y_i - \hat{y}_i$$

Vertical distance between the observed target and the fitted prediction.

LEAST SQUARES FIT

$$\hat{\theta} = \arg \min_{\theta} \sum_{i=1}^n e_i^2$$

Squaring makes large errors count more and prevents positive and negative errors from cancelling.



Read LinearRegression with the four questions

19

MODEL

$$\hat{y} = \theta_0 + \sum_{j=1}^8 \theta_j x_j$$

A linear family: one intercept plus one coefficient for each feature.

LEARNED PARAMETERS

$$\hat{\theta}_0, \dots, \hat{\theta}_8$$

Stored after `fit` as `intercept_` and `coef_`.

HYPERPARAMETER

`fit_intercept=True`

Chosen before fitting; controls whether the model includes an intercept.

LOSS / CRITERION

$$\sum_i (y_i - \hat{y}_i)^2$$

Ordinary least squares chooses coefficients with the smallest residual sum of squares.

THE TEMPLATE ANSWER

choose the hyperparameter → **fit the learned parameters** → **obtain one fitted linear rule**



Fit and predict in scikit-learn

20

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import root_mean_squared_error
```

```
model = LinearRegression()
```

```
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

```
rmse = root_mean_squared_error(y_test, y_pred)
```

`fit` sees targets. `predict` sees features only.

1 Construct

Choose the model family.

2 Fit

Learn coefficients from training examples.

3 Predict

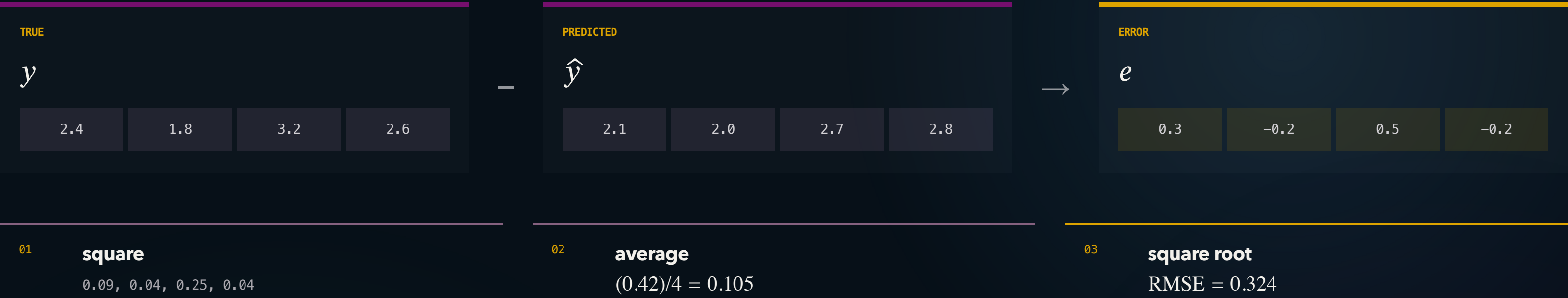
Produce one value for every test row.

4 Evaluate

Summarize the prediction errors.



Many errors become one RMSE



The scikit-learn estimator interface

A shared contract turns different learning rules into reusable prediction objects.



One contract for every prediction model

CONSTRUCT

```
model = Estimator(...)
```

Choose the model and its settings.

```
model.fit(X, y)
```

LEARN

fitted estimator

Store patterns learned from the training data.

```
model.predict(X_new)
```

APPLY

prediction vector

Return one prediction for each new row.

```
LinearRegression()
```

```
KNeighborsRegressor()
```

```
YourRegressor()
```

can share the same interface

The contract specifies **how we use a model**. Each estimator supplies its own learning and prediction rules.



The estimator as an octopus

24

CONFIGURATION

attributes before fit

`fit_intercept` `positive`

ONE PYTHON OBJECT

estimator

`model`

LEARNED STATE

attributes after fit

`coef_` `intercept_`

1 **fit(X, y)**
writes learned state

1

3 **score(X, y)**
evaluates

3

4 **get_params()**
reads configuration

4

2

2 **predict(X_new)**
reads learned state

5

5 **set_params(...)**
changes configuration

One object stores the state. Its methods provide controlled ways to use or update that state.

Different estimator classes may add more "tentacles," while the common sklearn methods keep familiar names.

Octopus illustration: [Jason J. Patterson](#) / [Open Clip Art Library](#), CC0.



One format, many regression algorithms

25

SCIKIT-LEARN REGRESSOR ESTIMATOR

Estimator(hyperparameters)

configuration

fit(X, y) returns self

learn from data

predict(X_new) returns y_pred

apply learned state

learned_attribute_

fitted state

each class fills
in
its own algorithm

LINEAR REGRESSION

LinearRegression

coef_

learn coefficients

NEAREST NEIGHBORS

KNeighborsRegressor

algorithm-specific state

store training cases

CUSTOM REGRESSOR

YourRegressor

learned_state_

supply your own rule

The “template” is an API contract.

It standardizes names, inputs, outputs, and object state. It does not prescribe the learning algorithm.



Class, estimator, and fitted estimator

PYTHON CLASS

LinearRegression

```
class LinearRegression(...):  
    def __init__(...)  
    def fit(self, X, y)  
    def predict(self, X)
```

Defines which parameters, methods, and fitted attributes an object may have.

call the class

creates

ESTIMATOR INSTANCE

```
model = LinearRegression(  
    fit_intercept=True)
```

available now

`fit_intercept`

not learned yet

`coef_`

A configured Python object, ready to receive data.

`model.fit(X, y)`

updates

FITTED ESTIMATOR

`model`

configuration

`fit_intercept`

learned from data

`coef_`

learned from data

`intercept_`

The same object now contains fitted state.

`model.fit(X, y)` is `model`

`True`

`fit` conventionally returns `self`.



LinearRegression as a running example

27

```
import numpy as np
from sklearn.linear_model import LinearRegression
```

```
X_train = np.array([
    [1., 0.],
    [0., 1.],
    [1., 1.],
    [2., 1.],
])
y_train = np.array([3., 2., 4., 6.])
```

```
model = LinearRegression(fit_intercept=True)
```

`X_train`

four samples, two features

Each row describes one training case.

`y_train`

four known targets

Position j matches row j of `X_train`.

`model`

an unfitted estimator

It knows its settings but has not learned a line.

`LinearRegression(...)` creates a Python object. Training happens only when we call `fit`.



Hyperparameters configure fitting

28

```
model = LinearRegression(  
    fit_intercept=True,  
    positive=False,  
)
```

```
model.get_params()["fit_intercept"]  
# True
```

```
model.set_params(positive=True)  
# LinearRegression(positive=True)
```

`init(...)`

records hyperparameters

These settings describe how a future fit should behave.

`get_params()`

returns constructor settings

Scikit-learn calls them estimator “parameters” in its API.

`set_params(...)`

changes hyperparameters

Fit the estimator again after changing a setting.

Training data do not belong in `init`.

X and y enter through `fit(X, y)`.



fit learns from paired rows of X and y

INPUT

`X_train`

```
[[1., 0.],  
 [0., 1.],  
 [1., 1.],  
 [2., 1.]]
```

`shape (4, 2)`

MATCHED TARGETS

`y_train`

```
[3., 2., 4., 6.]
```

`shape (4,)`

```
model.fit(X_train,  
          y_train)
```

UPDATED OBJECT

`model`

learned coefficients now live inside the estimator

returned is `model # True`

`X_train[j]` and `y_train[j]` form one supervised example.

`fit` returns `self`, which makes `LinearRegression().fit(X, y)` valid syntax.



Fitted attributes expose the learned equation

```
model = LinearRegression().fit(X_train, y_train)
```

```
model.coef_  
# array([2., 1.])
```

```
model.intercept_  
# 1.0
```

```
model.n_features_in_  
# 2
```

LEARNED PREDICTION RULE

$$\hat{y} = 1.0 + 2.0x_1 + 1.0x_2$$

coef_	one coefficient per feature
intercept_	the fitted intercept
n_features_in_	number of features seen by fit

The trailing underscore marks an attribute created from data. Accessing `coef_` before `fit` fails because it does not exist yet.



predict applies the fitted equation row by row

```
X_new = np.array([
    [0., 2.],
    [3., 0.],
])

y_pred = model.predict(X_new)
# array([3., 7.])
```

```
X_new.shape      (2, 2)
y_pred.shape      (2,)
```

`predict` returns one value for each input row and preserves the row order.

ROW 0	[0., 2.]	$1 + 2(0) + 1(2)$	3.0
ROW 1	[3., 0.]	$1 + 2(3) + 1(0)$	7.0

Feature order must match training.

Column 0 still means x_1 and column 1 still means x_2 .



Configuration and learned state

HYPERPARAMETER

`fit_intercept=True`

A constructor setting chosen before training. It controls how `fit` behaves.

LEARNED MODEL PARAMETER

$\hat{\theta}$

The mathematical quantity estimated from `X_train` and `y_train`.

FITTED ATTRIBUTE

`coef_, intercept_`

The Python attributes that store the learned parameter after `fit`.

`fit_intercept`

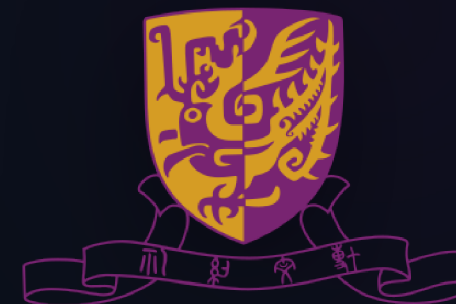
controls fitting

$\hat{\theta}$

stored inside the object

`coef_ + intercept_`

Hyperparameters control learning. Learned parameters are stored as fitted attributes.



Anatomy of a regressor estimator

```
from sklearn.base import BaseEstimator, RegressorMixin
from sklearn.utils.validation import check_is_fitted

class RegressorTemplate(RegressorMixin, BaseEstimator):
    def __init__(self, setting=1):
        self.setting = setting

    def fit(self, X, y):
        self.learned_value_ = ...
        return self

    def predict(self, X):
        check_is_fitted(self, "learned_value_")
        return ...
```

fit returns self, so `Estimator().fit(X, y)` is the fitted object.

`init`

stores settings

No training data and no fitted values.

`fit`

learns state

Learned public attributes end in `_`.

`predict`

uses fitted state

It never receives the unknown targets.



Recommender baselines: same template, new models

Keep the four questions and estimator lifecycle fixed; change only the prediction rule and learned state.



Rating triples match the estimator input contract

35

TRAINING TRIPLES

user	item	rating
1960	670	4
1346	152	4
785	1741	4

split columns

`X_train`

user-item pairs

```
[[1960, 670],  
 [1346, 152],  
 [ 785, 1741]]
```

shape: (n_samples, 2)

`y_train`

ratings

```
[4, 4, 4]
```

shape: (n_samples,)

`X_new[j] = (u, i)`

produces

`y_pred[j] = r_hat[u, i]`

same row, same position

`fit(X, y)` receives pairs and known ratings. `predict(X_new)` receives pairs only.



GlobalMeanRS: specify before coding

36

1 · MODEL

$$f_c(u, i) = c$$

The family contains every constant prediction rule.

2 · HYPERPARAMETERS

none

No setting is chosen before fitting.

3 · LOSS / CRITERION

$$J(c) = \sum_{(u, i, r) \in \square_{tr}} (r - c)^2$$

Compare candidate constants by their total squared error.

FIT · OPTIMIZE

$$\hat{\mu} = \arg \min_{c \in \mathbb{R}} \sum_{(u, i, r) \in \square_{tr}} (r - c)^2 = \frac{1}{|\square_{tr}|} \sum_{(u, i, r) \in \square_{tr}} r$$

4 · LEARNED PARAMETER AFTER `fit`

$$\hat{\mu}$$

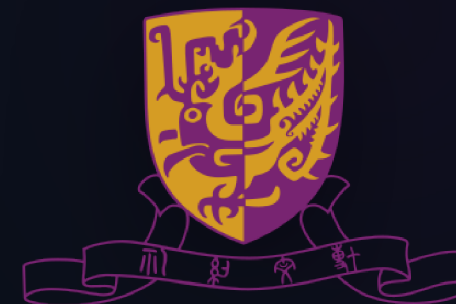
stored as `global_mean_` · `predict` returns it once per new row



Translate the specification into estimator methods

HYPERPARAMETERS	→	<code>init(...)</code>	These basic baselines have none, so an explicit constructor is unnecessary.
LEARNED PARAMETERS	→	<code>fit(X, y)</code>	Calculate them from training ratings and store them in attributes ending in <code>_</code> .
MODEL RULE	→	<code>predict(X_new)</code>	Read the fitted attributes and return one prediction for each input row.
LOSS / CRITERION	→	explains the fitting rule	It tells us why the learned mean is the chosen parameter value.
<code>check_is_fitted(...)</code>	guards the boundary	<code>predict</code> should fail clearly when the learned parameters do not exist yet.	

First specify the mathematics. Then each method has one clear job.



Now implement GlobalMeanRS

38

```
import numpy as np
from sklearn.base import BaseEstimator, RegressorMixin
from sklearn.utils.validation import check_is_fitted
```

```
class GlobalMeanRS(RegressorMixin, BaseEstimator):
    def fit(self, X, y):
        self.global_mean_ = np.asarray(y).mean()
        return self

    def predict(self, X):
        check_is_fitted(self, "global_mean_")
        return np.full(len(X), self.global_mean_)
```

FIT

one learned value

`global_mean_` comes from `y`.

PREDICT

one value per row

`X` determines only how many predictions to return.

CHECK

fit must happen first

Otherwise scikit-learn raises `NotFittedError`.



One object moves through the full lifecycle

39

01 `model = GlobalMeanRS()`

configured, not fitted

02 `model.fit(X_train, y_train)`

learns `global_mean_`

03 `model.global_mean_`

inspect fitted state

04 `y_pred = model.predict(X_test)`

one prediction per test pair

SAME OBJECT

Configuration remains. Fitted attributes appear after `fit`.



UserMeanRS: specify before coding

40

1 · MODEL

$$f_a(u, i) = a_u$$

One candidate constant per user; use the global rule for an unseen user.

2 · HYPERPARAMETERS

none

The grouping and fallback rule are fixed in this definition.

3 · LOSS / CRITERION

$$J_u(c) = \sum_{i \in \square_u^r} (r_{ui} - c)^2$$

For each observed user, compare candidate constants by squared error.

FIT · OPTIMIZE FOR
EACH OBSERVED
USER u

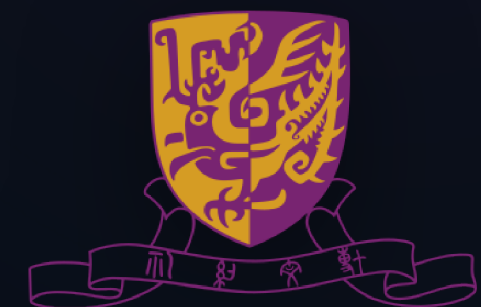
$$\hat{a}_u = \arg \min_{c \in \mathbb{R}} \sum_{i \in \square_u^{tr}} (r_{ui} - c)^2 = \frac{1}{|\square_u^{tr}|} \sum_{i \in \square_u^{tr}} r_{ui}$$

4 · LEARNED PARAMETERS AFTER `fit`

$$\widehat{\mu}, \{\hat{a}_u\}$$

stored as `global_mean_` and `user_means_` · retrieve by user ID





User mean learns a lookup vector

41

GROUP TRAINING RATINGS BY USER

u_0	4	2	3	mean = 3.0
-------	---	---	---	------------

u_1	5	4	3	mean = 4.0
-------	---	---	---	------------

u_2	2			mean = 2.0
-------	---	--	--	------------

store

FITTED ATTRIBUTES

```
global_mean_ = 3.3
```

```
user ID      0    1    2  
user_means_ = [3.0, 4.0, 2.0]
```

Only a user absent from training uses `global_mean_`.

Every observed user receives a mean. The global mean handles users absent from training.



Now implement UserMeanRS

```
class UserMeanRS(RegressorMixin, BaseEstimator):
    def fit(self, X, y):
        X, y = np.asarray(X), np.asarray(y)
        users = X[:, 0].astype(int)
        self.global_mean_ = y.mean()
        self.user_means_ = np.full(
            users.max() + 1, self.global_mean_
        )

        for user in set(users):
            ratings = y[users == user]
            self.user_means_[user] = ratings.mean()
        return self

    def predict(self, X):
        check_is_fitted(self, ["global_mean_", "user_means_"])
        users = np.asarray(X)[:, 0].astype(int)
        y_pred = np.full(len(users), self.global_mean_)
        valid = (users >= 0) & (users < len(self.user_means_))
        y_pred[valid] = self.user_means_[users[valid]]
        return y_pred
```

global_mean_

fallback learned by fit

user_means_

NumPy lookup vector learned by fit

users == user

select that user's ratings

valid

safe array indices or fallback



Teaching implementation: expects a two-column array with user ID in column 0.

User mean predicts by lookup

```
model = UserMeanRS().fit(X_train, y_train)
```

```
X_new = np.array([
    [0, 17], # user seen in training
    [71, 17], # user absent from training
])
```

```
model.predict(X_new)
# array([3.36, 3.62])
```

ROW 0 · USER 0

user_means_[0]

training ratings replaced the initial value

3.36

ROW 1 · USER 71

user_means_[71]

no training ratings, so the global mean remains

3.62

Values rounded from the STAT3009 Netflix training split.



ItemMeanRS: transfer the same template

44

1 · MODEL

$$f_b(u, i) = b_i$$

One candidate constant per item; use the global rule for an unseen item.

2 · HYPERPARAMETERS

none

The grouping and fallback rule are fixed in this definition.

3 · LOSS / CRITERION

$$J_i(c) = \sum_{u \in \square_i^{tr}} (r_{ui} - c)^2$$

For each observed item, compare candidate constants by squared error.

FIT · OPTIMIZE FOR
EACH OBSERVED
ITEM i

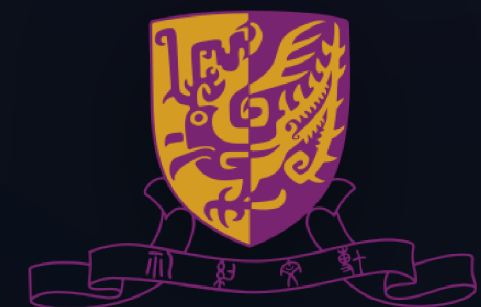
$$\hat{b}_i = \arg \min_{c \in \mathbb{R}} \sum_{u \in \square_i^{tr}} (r_{ui} - c)^2 = \frac{1}{|\square_i^{tr}|} \sum_{u \in \square_i^{tr}} r_{ui}$$

4 · LEARNED PARAMETERS AFTER `fit`

$$\hat{\mu}, \{\hat{b}_i\}$$

stored as `global_mean_` and `item_means_` · retrieve by item ID





What the base classes provide

45

YOUR CLASS

fit and predict

The recommender-specific learning rule lives here.

RegressorMixin

regressor identity and default score

The default regression score is R^2 .

BaseEstimator

get_params, set_params, representation

These support cloning and model-selection tools.

```
class MyRS(RegressorMixin, BaseEstimator):
```

mixin on the left, BaseEstimator on the right



Evaluation metrics stay outside the estimator

46

```
from sklearn.metrics import root_mean_squared_error
```

```
model = UserMeanRS()  
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)  
rmse = root_mean_squared_error(y_test, y_pred)
```

ESTIMATOR

learns and predicts

`fit` and `predict` define the shared model interface.

METRIC

judges predictions

The course reports RMSE so we call the metric explicitly.

Do not read `model.score(...)` as RMSE.

`RegressorMixin.score` returns R^2 by default, where larger is better.



One workflow can use every estimator

```
models = [  
    GlobalMeanRS(),  
    UserMeanRS(),  
]  
  
for model in models:  
    fitted_model = model.fit(X_train, y_train)  
    y_pred = fitted_model.predict(X_new)  
  
    print(type(model).__name__, y_pred.shape)
```

OUTPUT	
GlobalMeanRS	(m,)
UserMeanRS	(m,)

Same inputs	X_train, y_train, X_new
Same methods	fit and predict
Same output contract	one prediction per new row

Model-specific logic stays inside each class. Workflow code depends only on the shared estimator interface.



One interface—and one way to read every estimator

48

ASK FOUR QUESTIONS

`model`

`learned parameters`

`hyperparameters`

`loss / criterion`

then use

`fit(X, y)`

`predict(X_new)`

next lecture

ML II

Which fitted estimator will generalize?

Holdout validation and cross-validation compare candidates without using the final test labels.

For every new estimator: answer the four questions to understand it, then use the familiar `fit` / `predict` interface.

